National Institute of Business Management

Database Management 2 - Report

(LifeLineConnect)

Batch	:	26.1F
Course	:	HND-SE
Scopes	:	**Oracle PL/SQL + MongoDB**
Index	:	MAHNDSE261F-001 D.P.D Madushan

TABLE OF CONTENTS

Project Description 3

Deliverable Materials 3

System Design 3

System Diagram 4

Features and Functionalities 4

1. General UI 5
2. Camps 6
3. Appeals 7
4. Discussions 8
5. Resources, Medical Guidelines, Awareness Material 9

1. Donors 9
    1. Login 11
    2. Dashboard 12
    3. Eligibility 13
    4. Donor History 14
2. Organizing Committees 15
    1. Committee Dashboard 15
    2. Manage Camps 16
    3. Camp Operations 17
    4. Committee Staff 18
3. Blood Banks 19
    1. Blood Bank Dashboard 19
4. Webmaster 21
    1. Blood Bank Dashboard 21
    2. List of Users 22
    3. List of Blood Banks 23
    4. List of Organizing Committees 24
    5. Audit Logs 25
5. Export Reports 26
6. Backup 26
7. Pluggable Database 27

Project Description

LifeLineConnect is a fully featured platform built to ease the process of organizing, managing and conducting Blood Donation Camps. The system is mainly made to demonstrate the use cases of **Oracle Database PL/SQL and MongoDB**. For this specific scenario, we will be using

* Oracle Database 21c Express Edition (XE)
* MongoDB Atlas Cloud Edition

LifeLineConnect is divided into mainly four actors, all the features and functionalities are equally distributed among those actors.

1. Donor / Volunteers
    * Public users who are registered into the system for donating blood.
2. Organizing Committees
    * Those who organize blood donation camp and connect with blood banks.
3. Blood Banks
    * Responsible for managing blood transfers, blood inventories.
4. Webmaster
    * Administrative role, this user is responsible for managing and auditing the system.

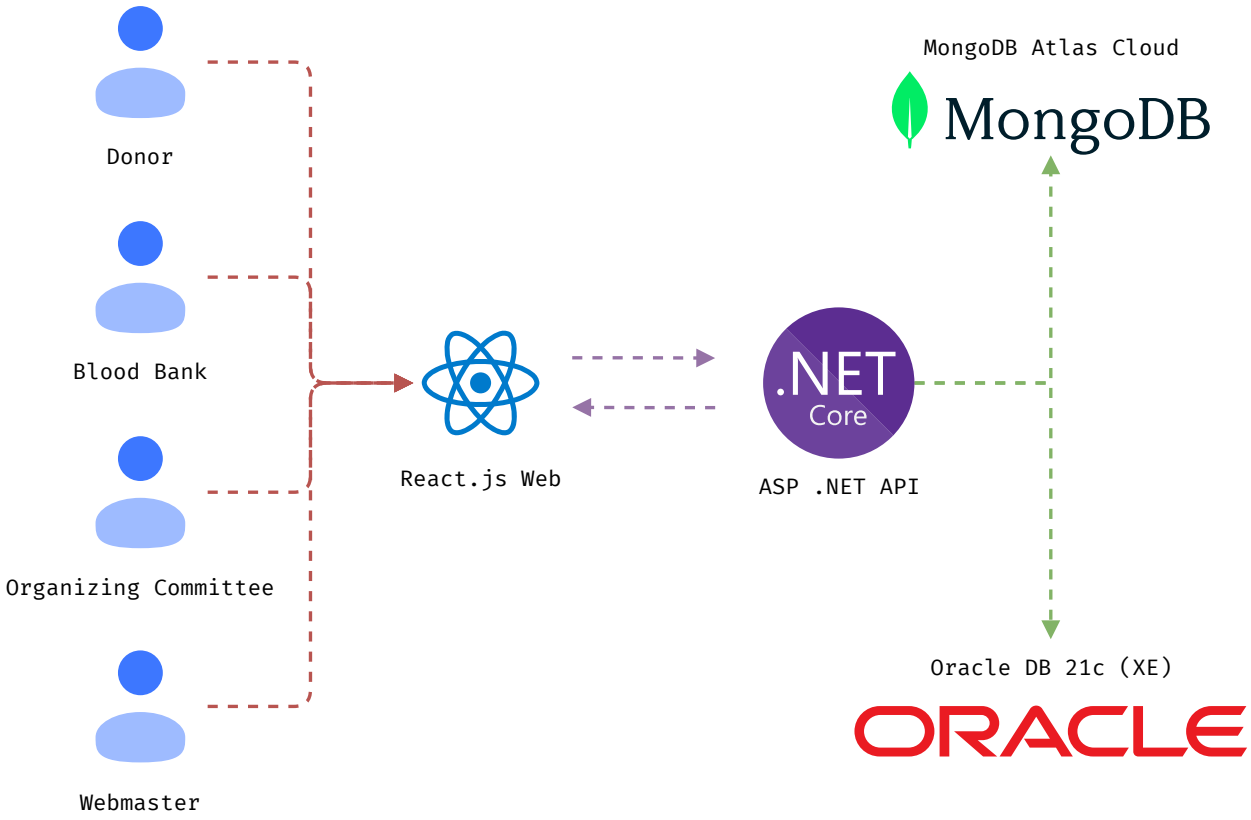
Deliverable Materials

ER Diagram	Google Drive
Presentation	
GitHub Repository	https://github.com/Danushka-Madushan/LifeLineConnect

System Design

The Application is made using an **ASP .NET C# API Backend** and a **React.js Frontend**. We are using **C# Oracle DB Connector** and **C# MongoDB Connector**. The reason I have choose to use this type of backend instead of using a general PHP backend is the ability to generate reports. C# have the native support for QuestPDF which is the report generation utility used in this specific application.

System Diagram



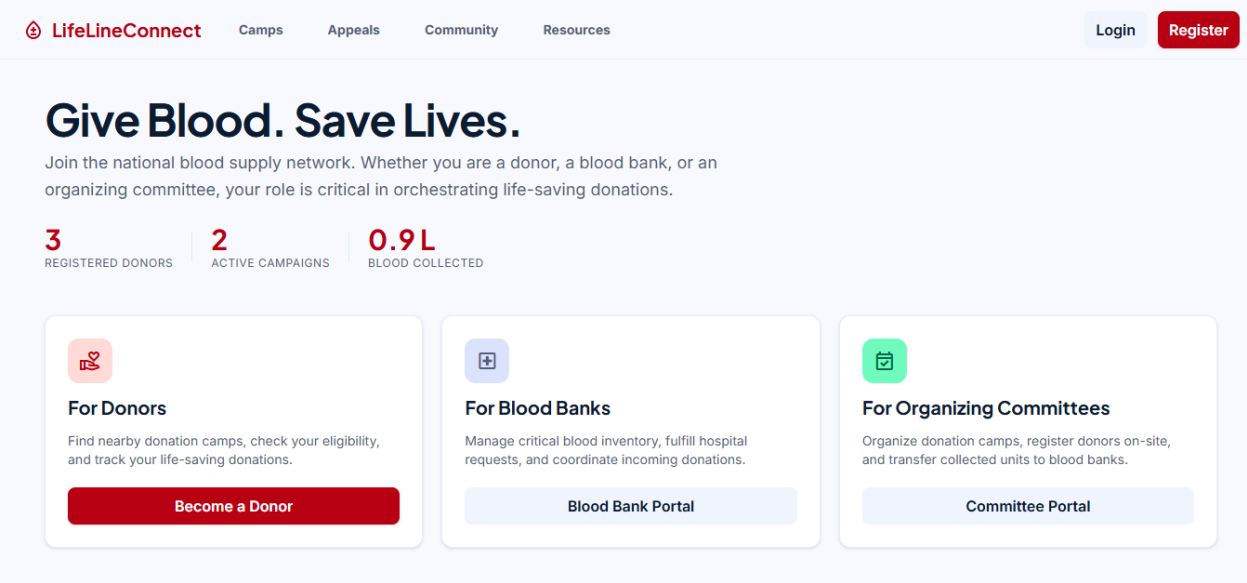
Features and Functionalities

Since the project is for demonstrating the usage of Oracle DB PL/SQL functionalities, I have removed all the inline SQL usages. Instead, every SQL query is executed or processed using the below PL SQL methods.

1. PL/SQL **Procedures**
2. PL/SQL **Functions**
3. PL/SQL **Cursors**
4. PL/SQL **Triggers**

For easy explanations I will be dividing each feature and functionality based on the main 4 actors. Each feature will be explained using the relevant PL/SQL code. There are some specific features such as Database Backup and Report Exporting. They will be explained within their own sections.

1. General UI

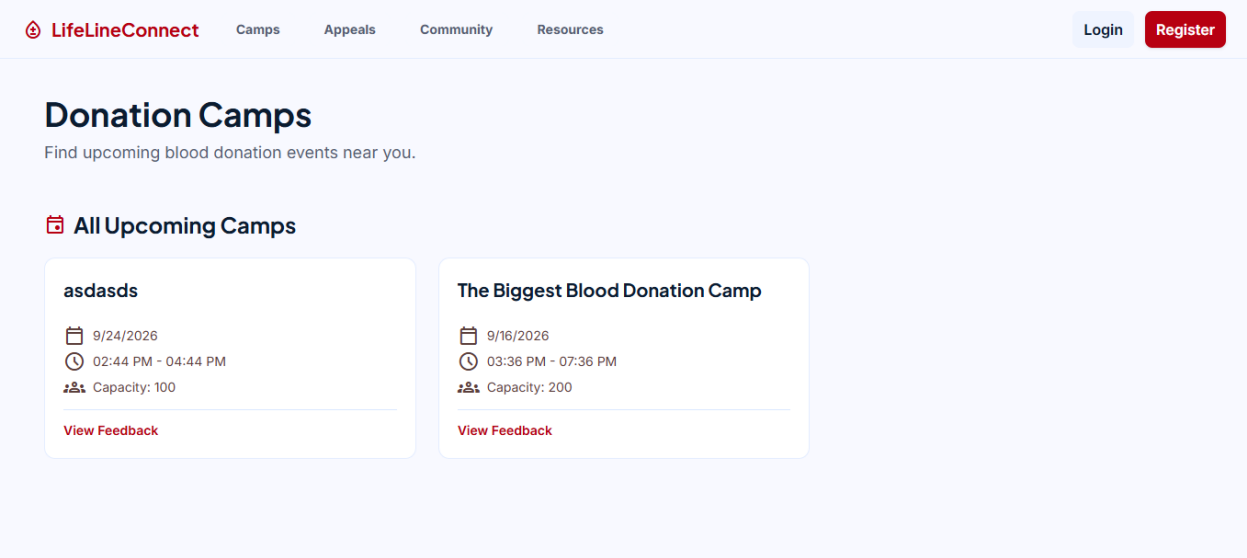


PLSQL

* Uses PL/SQL Function: **FN_GET_PUBLIC_STATS**
* This function returns a cursor containing a summary statistic of the count of active donors, the number of ongoing or published donation camps, and the total blood units collected.

```
CREATE OR REPLACE FUNCTION FN_GET_PUBLIC_STATS
RETURN SYS_REFCURSOR
AS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
        SELECT
            (SELECT COUNT(*) FROM DONOR WHERE STATUS = 'ACTIVE') AS
TOTAL_DONORS,
            (SELECT COUNT(*) FROM DONATION_CAMP WHERE STATUS IN
('PUBLISHED', 'ONGOING')) AS ACTIVE_CAMPS,
            (SELECT NVL(SUM(UNITS_COLLECTED), 0) FROM DONATION_RECORD WHERE
STATUS = 'SUBMITTED') AS TOTAL_UNITS
        FROM DUAL;
    RETURN v_cursor;
END FN_GET_PUBLIC_STATS;
```

2. Camps



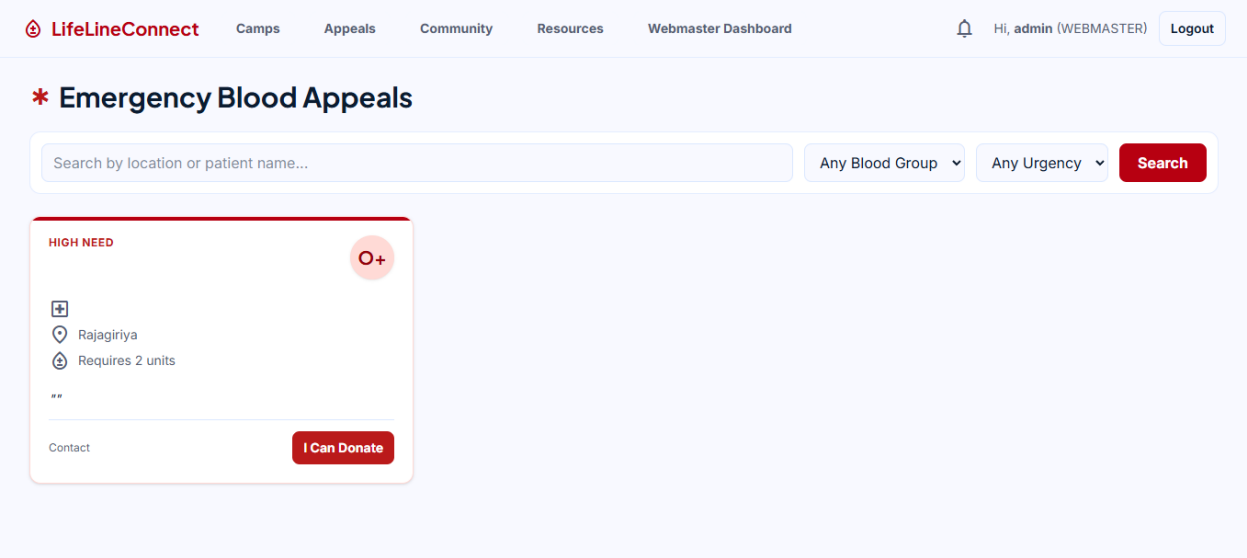
PLSQL

* Uses PL/SQL Function: **GET_PUBLIC_CAMPS**
* This function retrieves publicly visible donation camps and their venue coordinates for the home UI.

```
CREATE OR REPLACE PROCEDURE GET_PUBLIC_CAMPS (
    p_status        IN  VARCHAR2,
    p_lat           IN  NUMBER,
    p_lng           IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
IS
    v_sql VARCHAR2(4000);
BEGIN
    v_sql := 'SELECT c.CAMP_ID, c.COMMITTEE_ID, c.VENUE_ID, c.CAMP_TITLE,
c.CAMP_DESCRIPTION, ' ||
             'c.CAMP_DATE, c.START_TIME, c.END_TIME, c.CAPACITY, c.STATUS,
c.PUBLIC_VISIBLE, ' ||
             'v.LATITUDE, v.LONGITUDE ' ||
             'FROM DONATION_CAMP c ' ||
             'JOIN VENUE v ON c.VENUE_ID = v.VENUE_ID ' ||
             'WHERE c.PUBLIC_VISIBLE = ''Y'' ';

    IF p_status IS NOT NULL THEN
        v_sql := v_sql || ' AND c.STATUS = ''' || REPLACE(p_status, '''',
'''''') || ''' ';
    ELSE
        v_sql := v_sql || ' AND c.STATUS IN (''PUBLISHED'', ''ONGOING'',
''COMPLETED'') ';
```

```
    END IF;

    IF p_lat IS NOT NULL AND p_lng IS NOT NULL THEN
        v_sql := v_sql || ' ORDER BY SQRT(POWER(v.LATITUDE - (' || p_lat ||
'), 2) + POWER(v.LONGITUDE - (' || p_lng || '), 2)) ASC';
    ELSE
        v_sql := v_sql || ' ORDER BY c.CAMP_DATE DESC';
    END IF;

    OPEN p_result_cursor FOR v_sql;
END GET_PUBLIC_CAMPS;
```

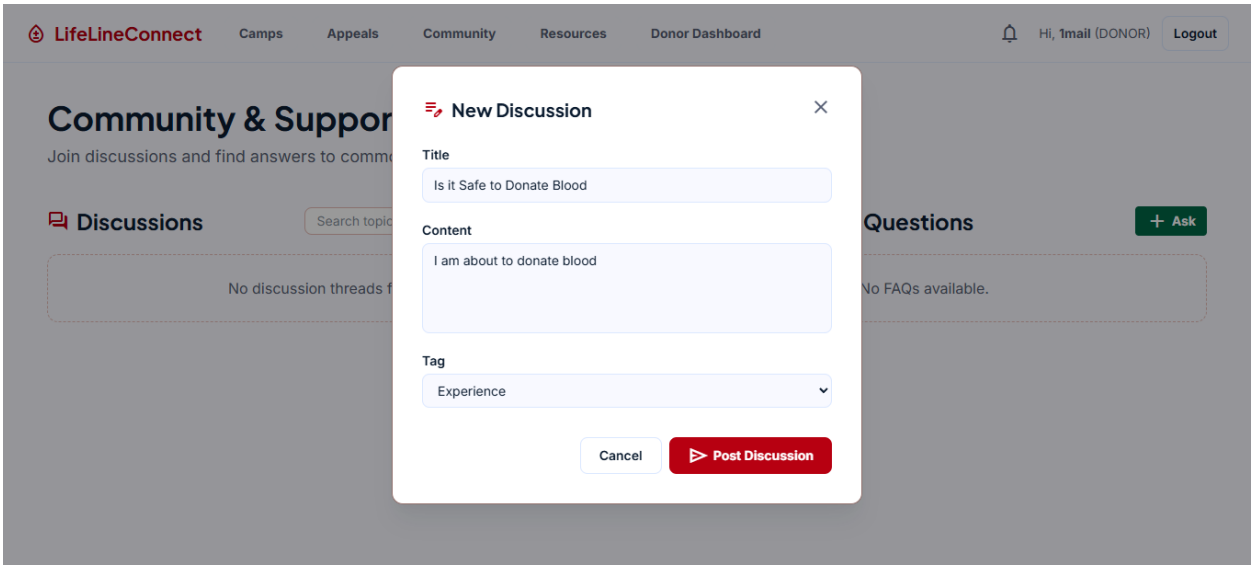
3. Appeals

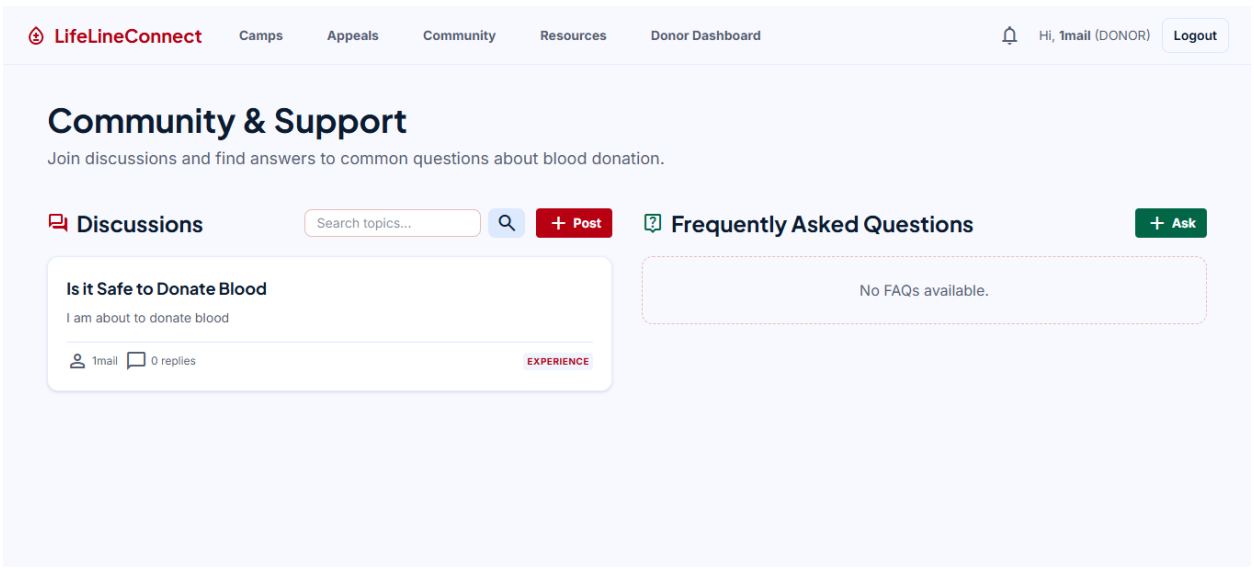


MongoDB

* Uses MongoDB: **db.emergencyAppeals.find()**
* This method retrieves all the documents from the emergency Appeals collection.

4. Discussions

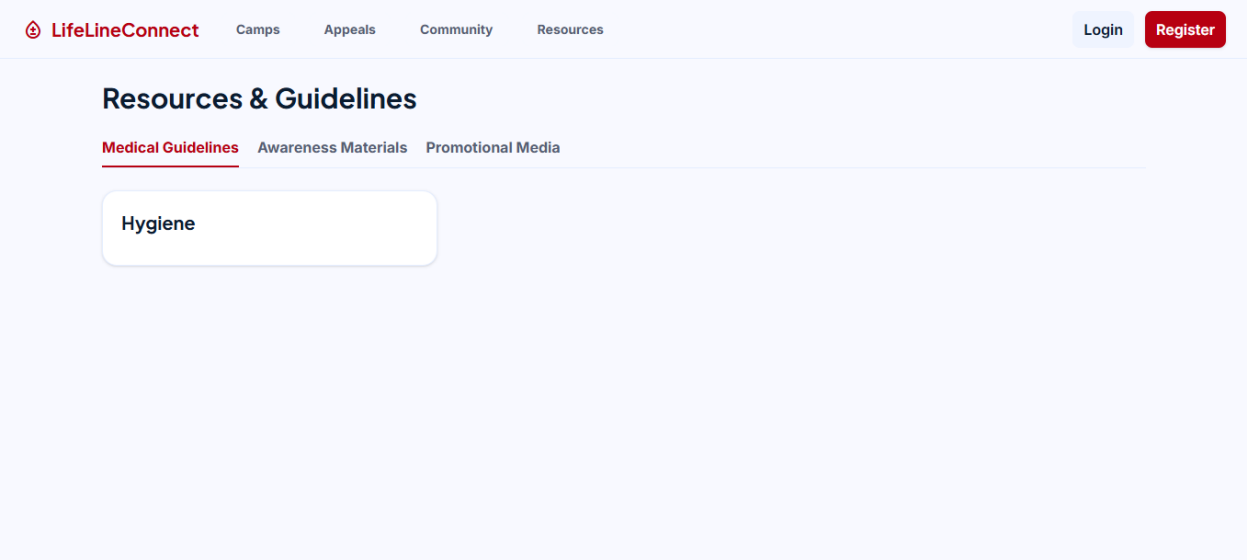




MongoDB

* Uses MongoDB: **db.communityThreads.insertOne()**
* This method inserts a new document into the communityThreads collection

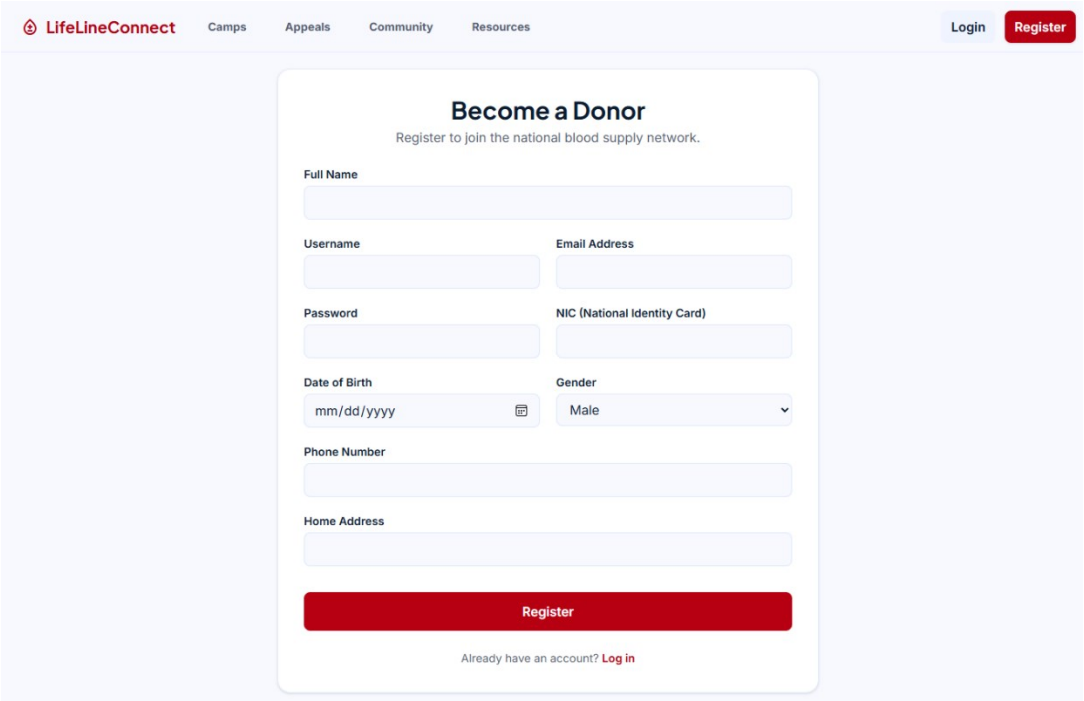
5. Resources, Medical Guidelines, Awareness Material



MongoDB

* Uses MongoDB: **db.medicalGuidelines.find(), db.awarenessMaterial.find()**
* This method gets all the medical guidelines awareness materials and promotional media via three mongodb collection calls and concatenate them into a one response.

1. Donors

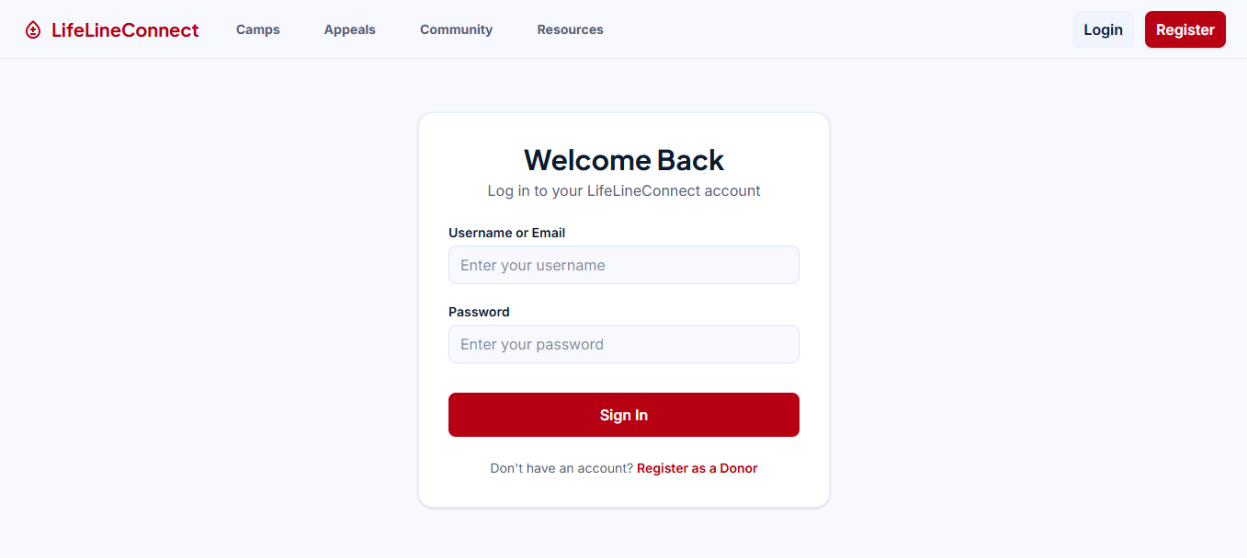


PLSQL

* Uses PL/SQL Procedure: **REGISTER_DONOR**
* This procedure registers a new donor to the system. It accepts set of input parameters and have 2 output parameters. output parameters will return **user_id** and **donor_id** on a successful account creation.

```
CREATE OR REPLACE PROCEDURE REGISTER_DONOR (
    p_username       IN  VARCHAR2,
    p_email          IN  VARCHAR2,
    p_password_hash  IN  VARCHAR2,
    p_full_name      IN  VARCHAR2,
    p_nic            IN  VARCHAR2,
    p_date_of_birth  IN  DATE,
    p_gender         IN  VARCHAR2,
    p_phone          IN  VARCHAR2,
    p_address        IN  VARCHAR2,
    p_user_id        OUT NUMBER,
    p_donor_id       OUT NUMBER
)
IS
BEGIN
    /* Create the APP_USER row */
    INSERT INTO APP_USER (USERNAME, EMAIL, PASSWORD_HASH, ACCOUNT_STATUS)
    VALUES (p_username, p_email, p_password_hash, 'ACTIVE')
    RETURNING USER_ID INTO p_user_id;

    /* Create the DONOR row */
    INSERT INTO DONOR (USER_ID, FULL_NAME, NIC, DATE_OF_BIRTH, GENDER,
PHONE, ADDRESS)
    VALUES (p_user_id, p_full_name, p_nic, p_date_of_birth, p_gender,
p_phone, p_address)
    RETURNING DONOR_ID INTO p_donor_id;

    /* Create the role link */
    INSERT INTO USER_ROLE_LINK (USER_ID, ROLE_CODE, DONOR_ID)
    VALUES (p_user_id, 'DONOR', p_donor_id);
END REGISTER_DONOR;
```

1. Login



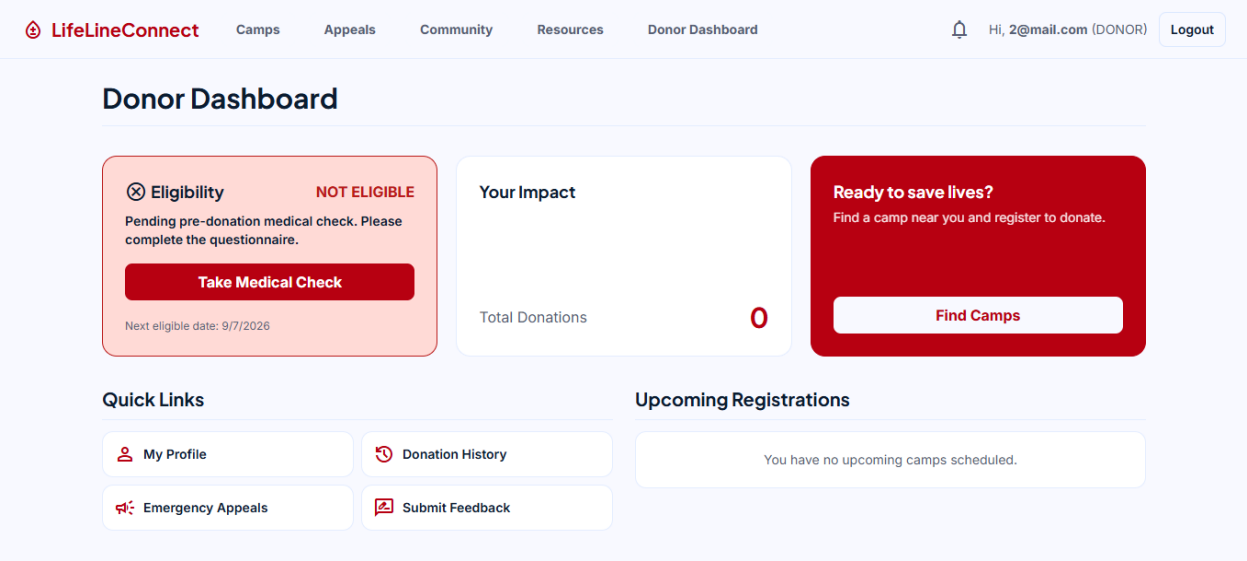
PLSQL

* Uses PL/SQL Procedure: **AUTHENTICATE_USER**
* This procedure uses on input parameter **Username** it can be ither the email or username. The input parameter will be matched with the username and email to verify the user.

```
CREATE OR REPLACE PROCEDURE AUTHENTICATE_USER (
    p_username        IN  VARCHAR2,
    p_user_id         OUT NUMBER,
    p_password_hash   OUT VARCHAR2,
    p_account_status  OUT VARCHAR2,
    p_role_code       OUT VARCHAR2
)
IS
    v_user APP_USER%ROWTYPE;
    v_role USER_ROLE_LINK.ROLE_CODE%TYPE;
BEGIN
    BEGIN
        /* email or username */
        SELECT * INTO v_user FROM APP_USER WHERE USERNAME = p_username OR
EMAIL = p_username;
        SELECT ROLE_CODE INTO v_role FROM USER_ROLE_LINK WHERE USER_ID =
v_user.USER_ID;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20009, 'Invalid username or
password.');
    END;

    IF v_user.ACCOUNT_STATUS != 'ACTIVE' THEN
```

```
        RAISE_APPLICATION_ERROR(-20008, 'Account is not active.');
    END IF;

    p_user_id := v_user.USER_ID;
    p_password_hash := v_user.PASSWORD_HASH;
    p_account_status := v_user.ACCOUNT_STATUS;
    p_role_code := v_role;

    /* Update last login timestamp */
    UPDATE APP_USER SET LAST_LOGIN_AT = SYSTIMESTAMP WHERE USER_ID =
p_user_id;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        p_user_id := -1;
        p_password_hash := NULL;
        p_account_status := NULL;
        p_role_code := NULL;
END AUTHENTICATE_USER;
```

2. Dashboard



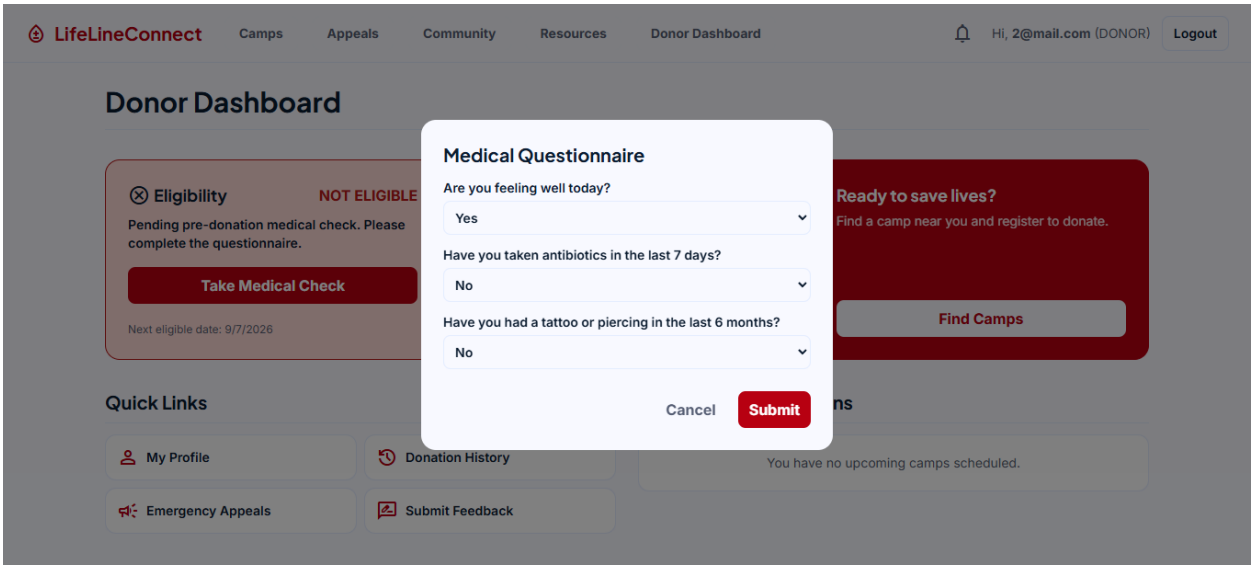
PLSQL

* Uses PL/SQL Procedure: **GET_DONOR_DASHBOARD**
* This procedure retrieves the dashboard data when the user is given as a input parameter.

```
CREATE OR REPLACE PROCEDURE GET_DONOR_DASHBOARD (
    p_user_id       IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
```

```
IS
    v_donor_id NUMBER;
BEGIN
    SELECT DONOR_ID INTO v_donor_id FROM DONOR WHERE USER_ID = p_user_id;

    OPEN p_result_cursor FOR
        SELECT
            (SELECT COUNT(*) FROM DONATION_RECORD WHERE DONOR_ID =
v_donor_id AND STATUS = 'SUBMITTED') AS TOTAL_DONATIONS,
            (SELECT COUNT(*) FROM CAMP_REGISTRATION cr
             JOIN DONATION_CAMP dc ON cr.CAMP_ID = dc.CAMP_ID
             WHERE cr.DONOR_ID = v_donor_id
               AND cr.REGISTRATION_STATUS = 'REGISTERED'
               AND dc.CAMP_DATE >= TRUNC(SYSDATE)) AS UPCOMING_CAMPS,
            (SELECT MAX(DONATION_DATE) FROM DONATION_RECORD WHERE DONOR_ID =
v_donor_id AND STATUS = 'SUBMITTED') AS LAST_DONATION_DATE
        FROM DUAL;
END GET_DONOR_DASHBOARD;
```

3. Eligibility



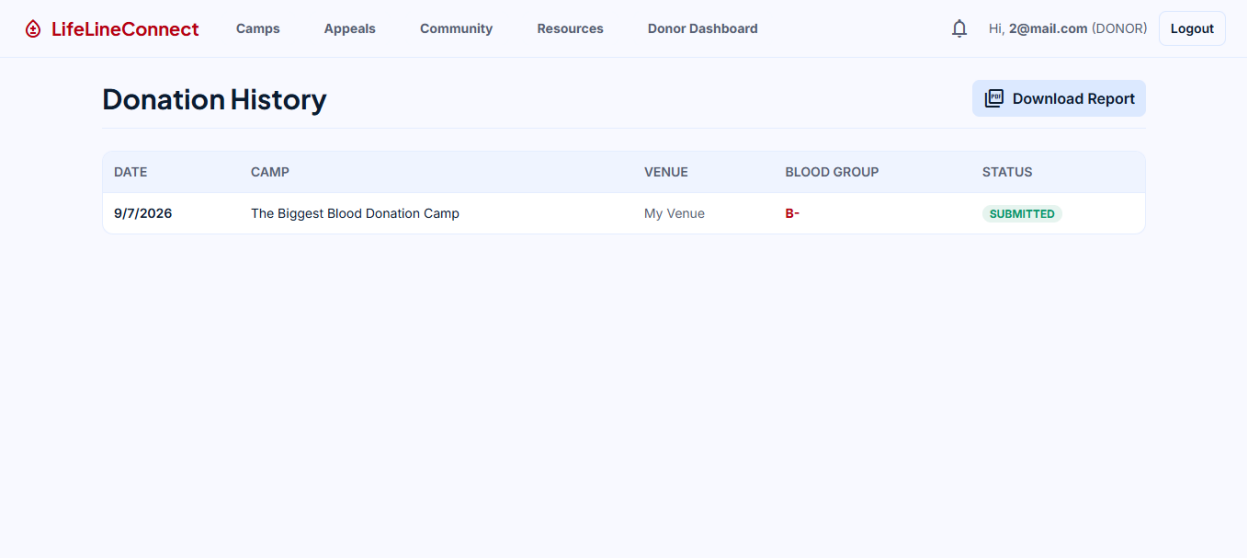
PLSQL

* Uses PL/SQL Procedure: **SUBMIT_MEDICAL_CHECK**
* This procedure accepts three parameters (medical questionaries) based on the answers the procedure returns if the user passe or fails the eligibility test.

```
CREATE OR REPLACE PROCEDURE SUBMIT_MEDICAL_CHECK(
    p_user_id IN NUMBER,
    p_feeling_well IN NUMBER,
```

```
    p_recent_antibiotics IN NUMBER,
    p_recent_tattoo IN NUMBER,
    p_status OUT VARCHAR2
)
IS
BEGIN
    IF p_feeling_well = 1 AND p_recent_antibiotics = 0 AND p_recent_tattoo =
0 THEN
        p_status := 'PASSED';
    ELSE
        p_status := 'FAILED';
    END IF;

    UPDATE DONOR
    SET MEDICAL_CHECK_STATUS = p_status,
        LAST_MEDICAL_CHECK_AT = CURRENT_TIMESTAMP
    WHERE USER_ID = p_user_id;

END SUBMIT_MEDICAL_CHECK;
```

4. Donor History



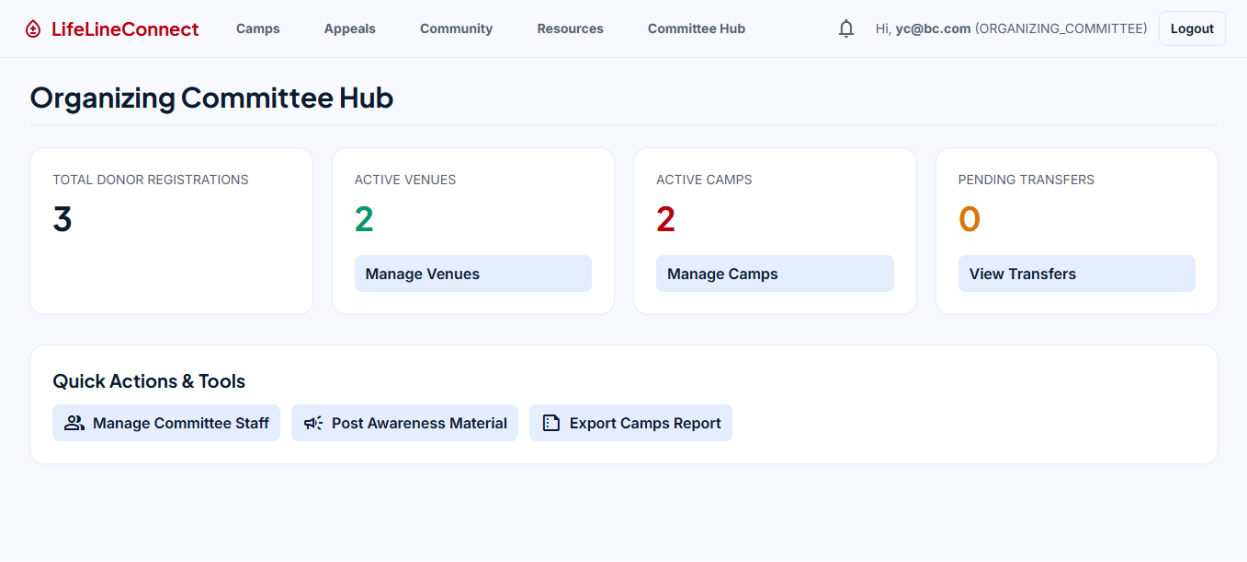
PLSQL

* Uses PL/SQL Procedure: **GET_DONOR_DONATION_HISTORY**
* This procedure accepts one input parameter and then it returns the related doner donations history

```
CREATE OR REPLACE PROCEDURE GET_DONOR_DONATION_HISTORY (
    p_user_id       IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
IS
    v_donor_id NUMBER;
BEGIN
    SELECT DONOR_ID INTO v_donor_id FROM DONOR WHERE USER_ID = p_user_id;

    OPEN p_result_cursor FOR
        SELECT dr.DONATION_ID, dr.DONATION_DATE, dr.BLOOD_GROUP,
               dr.UNITS_COLLECTED, dr.STATUS,
               dc.CAMP_ID, dc.CAMP_TITLE,
               v.VENUE_NAME
        FROM DONATION_RECORD dr
        JOIN DONATION_CAMP dc ON dr.CAMP_ID = dc.CAMP_ID
        JOIN VENUE v ON dc.VENUE_ID = v.VENUE_ID
        WHERE dr.DONOR_ID = v_donor_id
        ORDER BY dr.DONATION_DATE DESC;
END GET_DONOR_DONATION_HISTORY;
```

2. Organizing Committees

1. Committee Dashboard

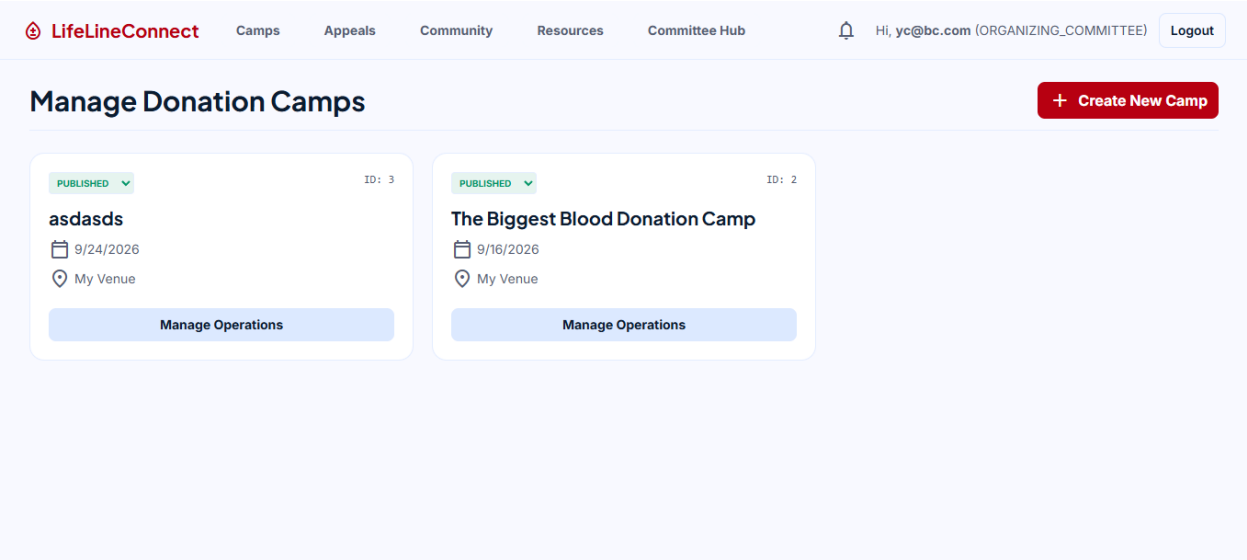


PLSQL

* Uses PL/SQL Procedure: **GET_COMMITTEE_DASHBOARD**
* This procedure accepts one input parameter and then it returns the related committee dashboard data.

```
CREATE OR REPLACE PROCEDURE GET_COMMITTEE_DASHBOARD (
    p_user_id       IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
IS
    v_committee_id ORGANIZING_COMMITTEE.COMMITTEE_ID%TYPE;
BEGIN
    SELECT COMMITTEE_ID INTO v_committee_id
    FROM USER_ROLE_LINK WHERE USER_ID = p_user_id AND ROLE_CODE =
'ORGANIZING_COMMITTEE';

    OPEN p_result_cursor FOR
        SELECT
            (SELECT COUNT(*) FROM DONATION_CAMP WHERE COMMITTEE_ID =
v_committee_id AND STATUS IN ('PUBLISHED', 'ONGOING')) AS ACTIVE_CAMPS,
            (SELECT COUNT(*) FROM DONATION_TRANSFER WHERE COMMITTEE_ID =
v_committee_id AND STATUS IN ('PREPARED', 'DISPATCHED', 'IN_TRANSIT')) AS
PENDING_TRANSFERS,
            (SELECT COUNT(*) FROM CAMP_REGISTRATION cr JOIN DONATION_CAMP dc
ON cr.CAMP_ID = dc.CAMP_ID WHERE dc.COMMITTEE_ID = v_committee_id) AS
TOTAL_REGISTRATIONS,
            (SELECT COUNT(*) FROM VENUE WHERE COMMITTEE_ID = v_committee_id
AND STATUS = 'ACTIVE') AS ACTIVE_VENUES
        FROM DUAL;
END GET_COMMITTEE_DASHBOARD;
```

2. Manage Camps



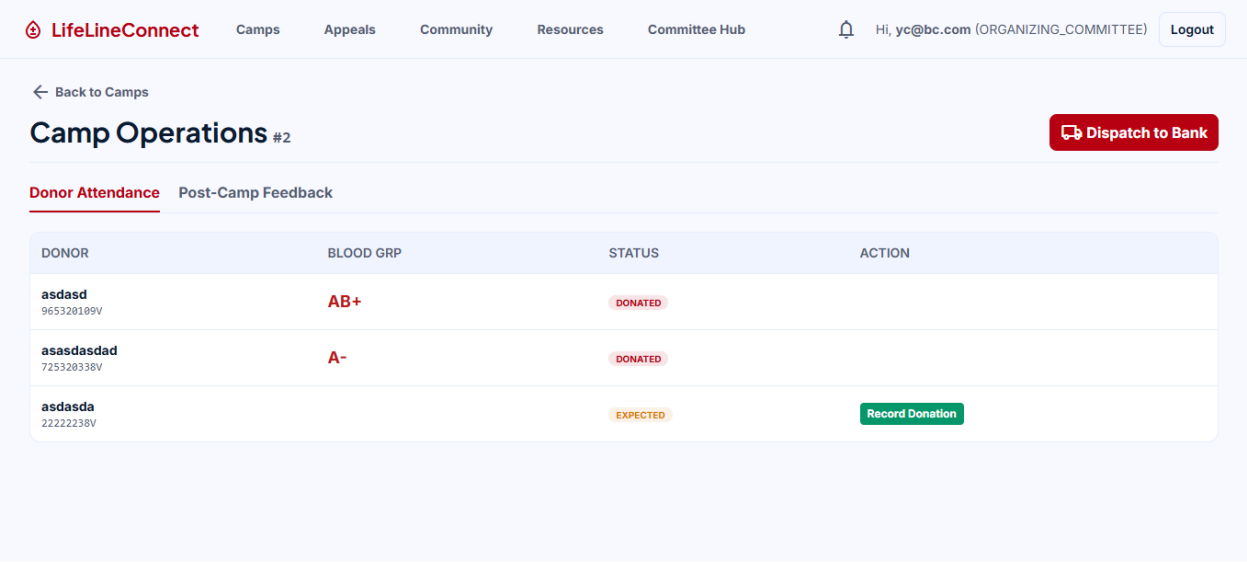
PLSQL

* Uses PL/SQL Procedure: **GET_COMMITTEE_CAMPS**

* This procedure accepts one input parameter, and it returns related committee organized camps.

```
CREATE OR REPLACE PROCEDURE GET_COMMITTEE_CAMPS (
    p_user_id       IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
IS
    v_committee_id ORGANIZING_COMMITTEE.COMMITTEE_ID%TYPE;
BEGIN
    SELECT COMMITTEE_ID INTO v_committee_id
    FROM USER_ROLE_LINK WHERE USER_ID = p_user_id AND ROLE_CODE =
'ORGANIZING_COMMITTEE';

    OPEN p_result_cursor FOR
        SELECT dc.CAMP_ID, dc.CAMP_TITLE, dc.CAMP_DATE,
               dc.START_TIME, dc.END_TIME, dc.CAPACITY,
               dc.STATUS, dc.PUBLIC_VISIBLE,
               v.VENUE_NAME
        FROM DONATION_CAMP dc
        JOIN VENUE v ON dc.VENUE_ID = v.VENUE_ID
        WHERE dc.COMMITTEE_ID = v_committee_id
        ORDER BY dc.CAMP_DATE DESC;
END GET_COMMITTEE_CAMPS;
```

3. Camp Operations



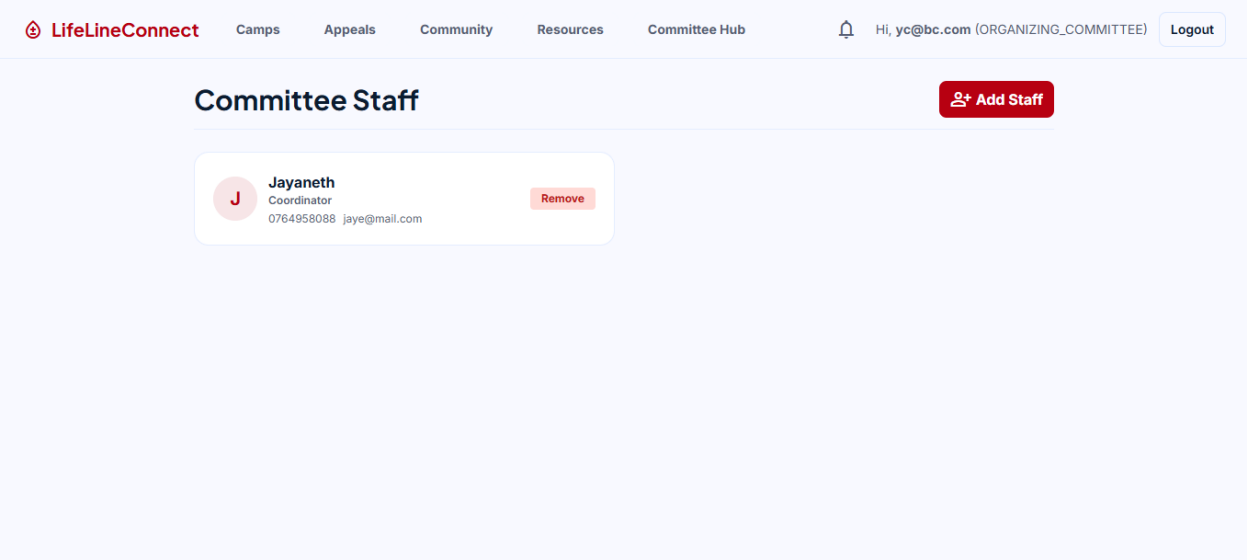
PLSQL

* Uses PL/SQL Procedure: **GET_CAMP_ATTENDANCE**

* This procedure accepts one input parameter and then it returns the related blood donation camp attendance.

```
CREATE OR REPLACE PROCEDURE GET_CAMP_ATTENDANCE (
    p_user_id       IN  NUMBER,
    p_camp_id       IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
IS
BEGIN
    OPEN p_result_cursor FOR
        SELECT cr.REGISTRATION_ID, cr.DONOR_ID,
               cr.REGISTRATION_STATUS, cr.ATTENDANCE_STATUS,
               d.FULL_NAME, d.NIC, d.BLOOD_GROUP,
               CASE WHEN EXISTS (
                   SELECT 1 FROM DONATION_RECORD dr
                   WHERE dr.REGISTRATION_ID = cr.REGISTRATION_ID AND
dr.STATUS IN ('DRAFT', 'SUBMITTED')
               ) THEN 1 ELSE 0 END AS HAS_DONATED
        FROM CAMP_REGISTRATION cr
        JOIN DONOR d ON cr.DONOR_ID = d.DONOR_ID
        WHERE cr.CAMP_ID = p_camp_id
        ORDER BY cr.REGISTERED_AT;
END GET_CAMP_ATTENDANCE;
```

4. Committee Staff



PLSQL

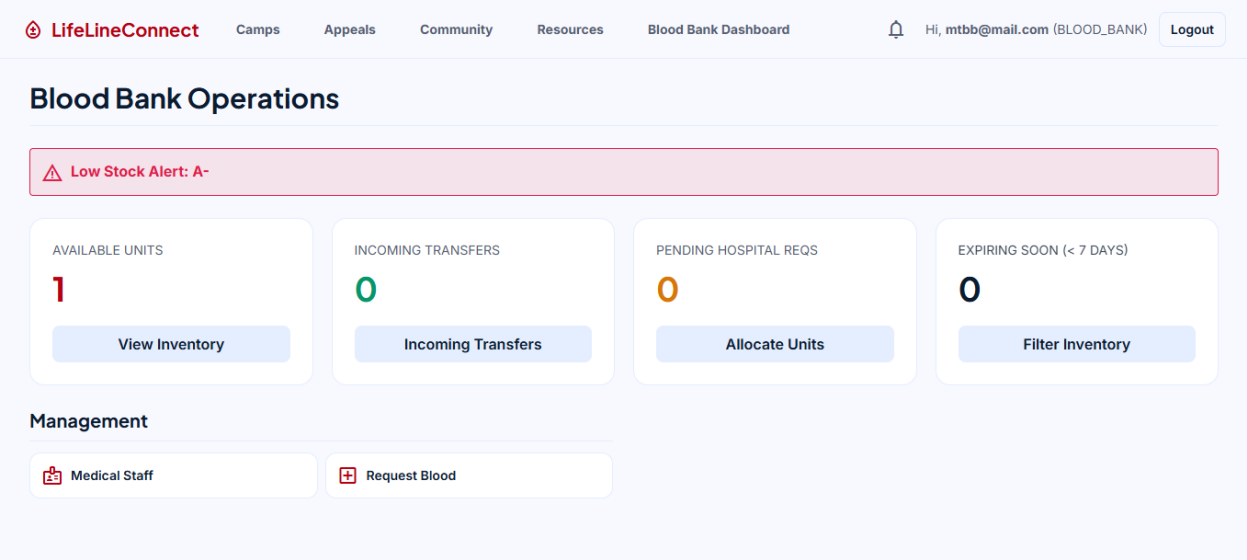
* Uses PL/SQL Procedure: **GET_COMMITTEE_STAFF**

* This procedure accepts one input parameter and then it returns the related committee staff data.

```
CREATE OR REPLACE PROCEDURE GET_COMMITTEE_STAFF (
    p_user_id       IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
IS
    v_committee_id ORGANIZING_COMMITTEE.COMMITTEE_ID%TYPE;
BEGIN
    SELECT COMMITTEE_ID INTO v_committee_id
    FROM USER_ROLE_LINK WHERE USER_ID = p_user_id AND ROLE_CODE =
'ORGANIZING_COMMITTEE';

    OPEN p_result_cursor FOR
        SELECT sm.STAFF_ID, sm.FULL_NAME, sm.POSITION_TITLE,
               sm.PHONE, sm.EMAIL,
               csa.ASSIGNED_FROM, csa.STATUS
        FROM COMMITTEE_STAFF_ASSIGNMENT csa
        JOIN STAFF_MEMBER sm ON csa.STAFF_ID = sm.STAFF_ID
        WHERE csa.COMMITTEE_ID = v_committee_id
        ORDER BY csa.ASSIGNED_FROM DESC;
END GET_COMMITTEE_STAFF;
```

3. Blood Banks

1. Blood Bank Dashboard



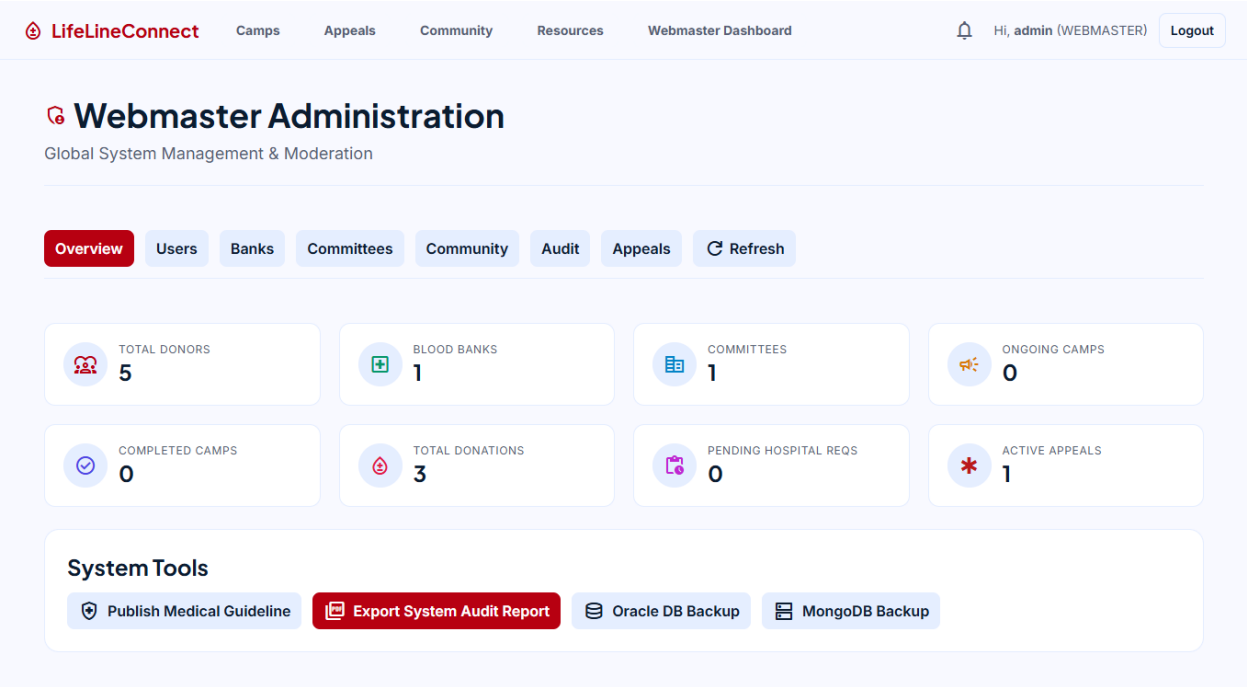
PLSQL

* Uses PL/SQL Procedure: **GET_BANK_DASHBOARD**
* This procedure accepts one input parameter, and it returns related bank dashboard data.

```
CREATE OR REPLACE PROCEDURE GET_BANK_DASHBOARD (
    p_user_id       IN  NUMBER,
    p_result_cursor OUT SYS_REFCURSOR
)
IS
    v_bank_id BLOOD_BANK.BLOOD_BANK_ID%TYPE;
BEGIN
    SELECT BLOOD_BANK_ID INTO v_bank_id
    FROM USER_ROLE_LINK WHERE USER_ID = p_user_id AND ROLE_CODE =
'BLOOD_BANK';

    OPEN p_result_cursor FOR
        SELECT
            (SELECT COUNT(*) FROM BLOOD_UNIT WHERE BLOOD_BANK_ID = v_bank_id
AND STATUS = 'AVAILABLE') AS TOTAL_UNITS,
            (SELECT COUNT(*) FROM DONATION_TRANSFER WHERE BLOOD_BANK_ID =
v_bank_id AND STATUS IN ('DISPATCHED', 'IN_TRANSIT')) AS INCOMING_TRANSFERS,
            (SELECT COUNT(*) FROM HOSPITAL_BLOOD_REQUEST WHERE BLOOD_BANK_ID
= v_bank_id AND STATUS = 'PENDING') AS PENDING_REQUESTS,
            (SELECT COUNT(*) FROM BLOOD_UNIT WHERE BLOOD_BANK_ID = v_bank_id
AND STATUS = 'AVAILABLE' AND EXPIRY_DATE <= TRUNC(SYSDATE) + 7) AS
EXPIRING_SOON,
            (SELECT LISTAGG(BLOOD_GROUP, ',') WITHIN GROUP (ORDER BY
BLOOD_GROUP) FROM (
                SELECT BLOOD_GROUP FROM BLOOD_UNIT
                WHERE BLOOD_BANK_ID = v_bank_id AND STATUS = 'AVAILABLE'
                GROUP BY BLOOD_GROUP
                HAVING COUNT(*) < 10
            )) AS LOW_STOCK_GROUPS
        FROM DUAL;
END GET_BANK_DASHBOARD;
```

4. Webmaster

1. Blood Bank Dashboard



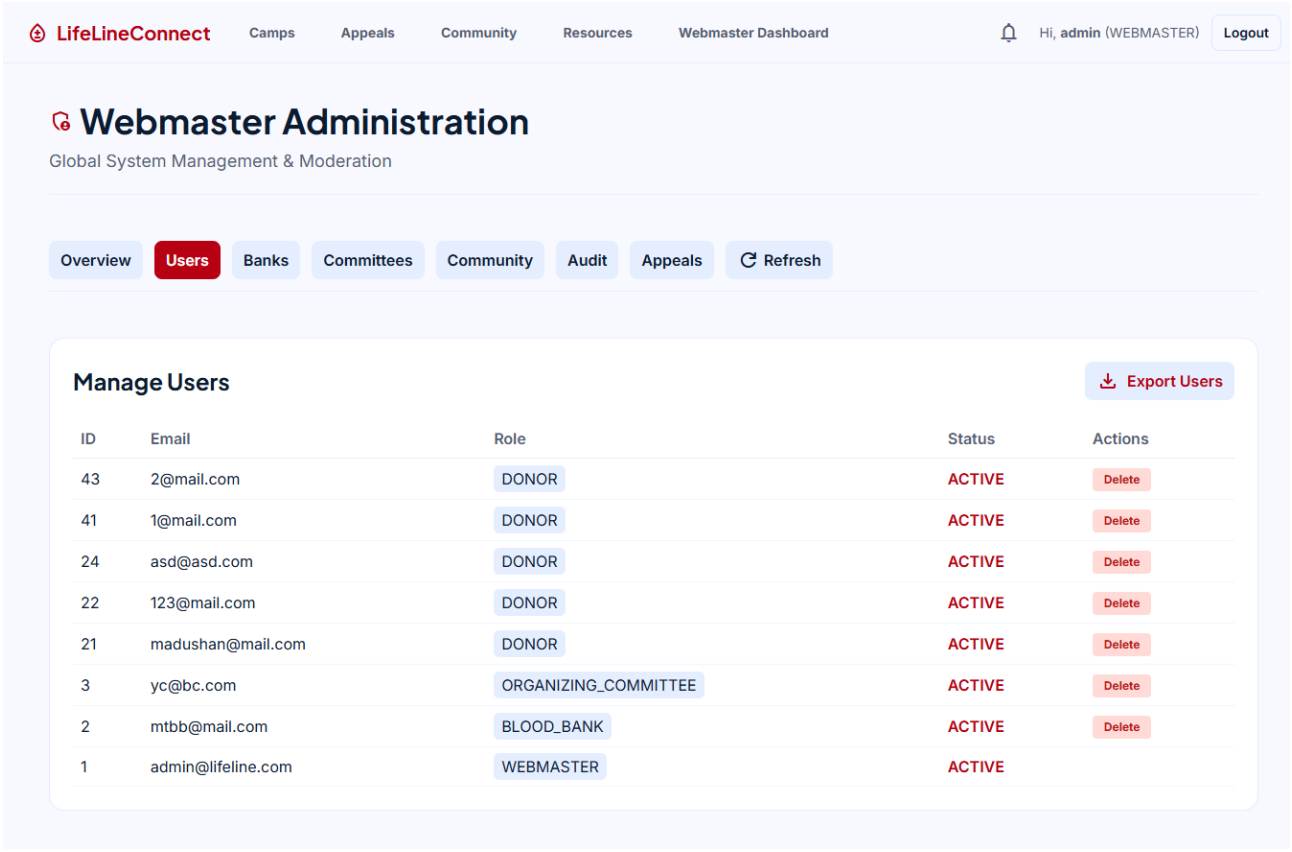
PLSQL

* Uses PL/SQL Procedure: **GET_WEBMASTER_DASHBOARD**
* This procedure accepts one input parameter, and it returns related webmaster dashboard data.

```
CREATE OR REPLACE PROCEDURE GET_WEBMASTER_DASHBOARD (
    p_result_cursor OUT SYS_REFCURSOR
)
IS
BEGIN
    OPEN p_result_cursor FOR
        SELECT
            (SELECT COUNT(*) FROM DONOR WHERE STATUS = 'ACTIVE') AS
TOTAL_DONORS,
            (SELECT COUNT(*) FROM BLOOD_BANK WHERE STATUS = 'ACTIVE') AS
TOTAL_BANKS,
            (SELECT COUNT(*) FROM ORGANIZING_COMMITTEE WHERE STATUS =
'ACTIVE') AS TOTAL_COMMITTEES,
            (SELECT COUNT(*) FROM DONATION_CAMP WHERE STATUS = 'ONGOING') AS
ONGOING_CAMPS,
            (SELECT COUNT(*) FROM DONATION_CAMP WHERE STATUS = 'COMPLETED')
AS COMPLETED_CAMPS,
```

```
           (SELECT COUNT(*) FROM DONATION_RECORD WHERE STATUS =
'SUBMITTED') AS TOTAL_DONATIONS,
           (SELECT COUNT(*) FROM HOSPITAL_BLOOD_REQUEST WHERE STATUS =
'PENDING') AS PENDING_REQUESTS
       FROM DUAL;
END GET_WEBMASTER_DASHBOARD;
```

2. List of Users



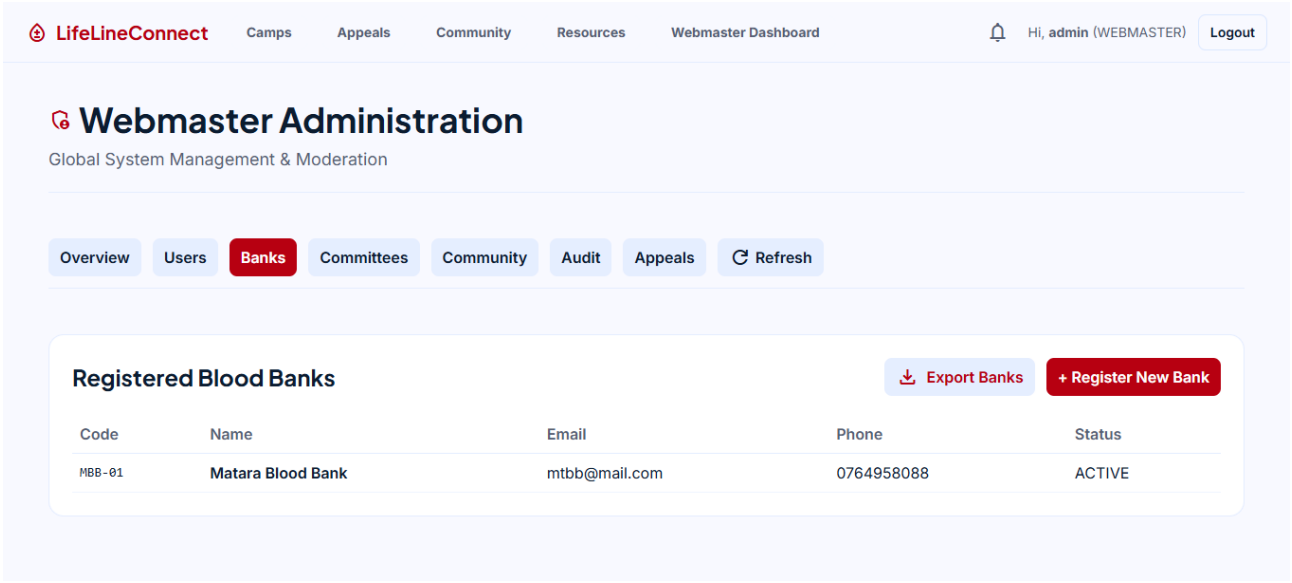
PLSQL

* Uses PL/SQL Procedure: **FN_GET_ALL_USERS**
* This procedure accepts one input parameter, and it returns related webmaster user data.

```
CREATE OR REPLACE FUNCTION FN_GET_ALL_USERS
RETURN SYS_REFCURSOR
IS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
        SELECT u.USER_ID, u.EMAIL, r.ROLE_CODE AS ROLE, u.ACCOUNT_STATUS AS
STATUS,
               u.CREATED_AT, u.LAST_LOGIN_AT AS LAST_LOGIN
```

```
        FROM APP_USER u
        JOIN USER_ROLE_LINK r ON u.USER_ID = r.USER_ID
        ORDER BY u.CREATED_AT DESC;
    RETURN v_cursor;
END FN_GET_ALL_USERS;
```

3. List of Blood Banks

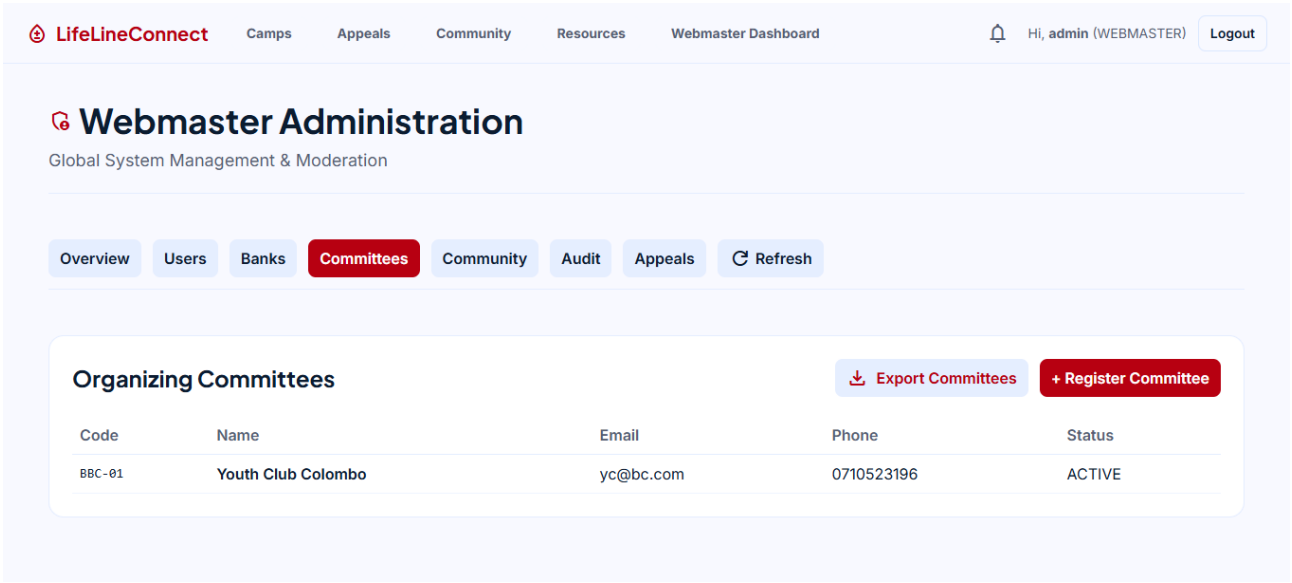


PLSQL

* Uses PL/SQL Procedure: **FN_GET_ALL_BANKS**
* This procedure accepts one input parameter, and it returns related webmaster bank data.

```
CREATE OR REPLACE FUNCTION FN_GET_ALL_BANKS
RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR SELECT BLOOD_BANK_ID, BANK_CODE, BANK_NAME, PHONE,
EMAIL, ADDRESS, STATUS FROM BLOOD_BANK ORDER BY BLOOD_BANK_ID DESC;
  RETURN v_cursor;
END;
```

4. List of Organizing Committees

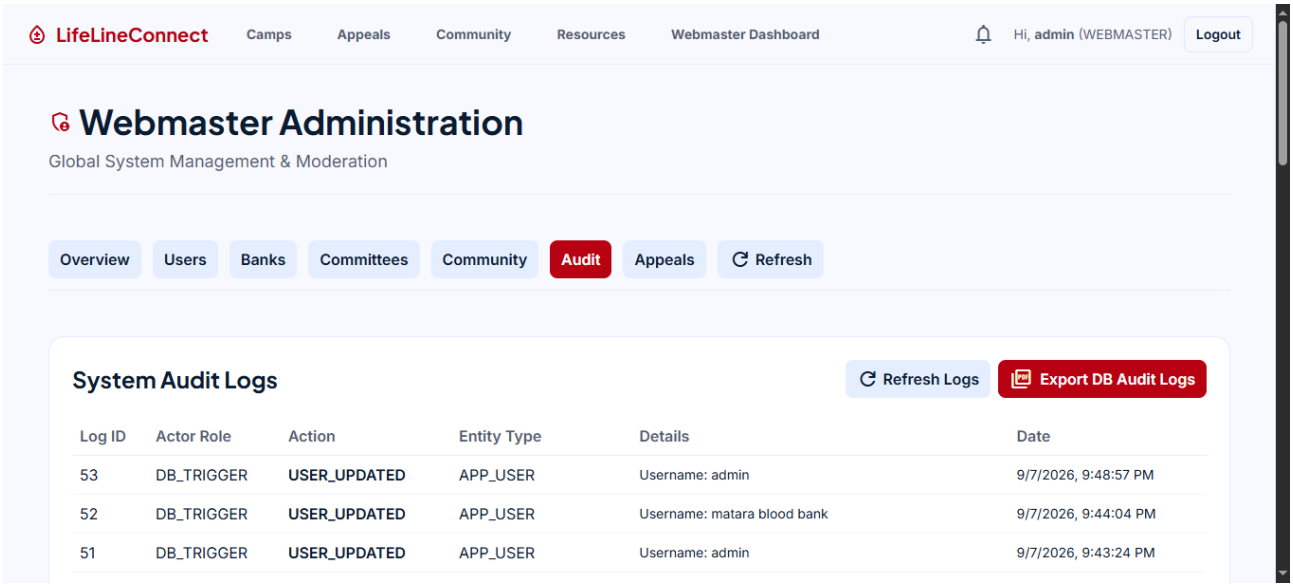


PLSQL

* Uses PL/SQL Procedure: **FN_GET_ALL_COMMITTEES**
* This procedure accepts one input parameter, and it returns related webmaster organizing data.

```
CREATE OR REPLACE FUNCTION FN_GET_ALL_COMMITTEES
RETURN SYS_REFCURSOR
IS
  v_cursor SYS_REFCURSOR;
BEGIN
  OPEN v_cursor FOR SELECT COMMITTEE_ID, COMMITTEE_CODE, COMMITTEE_NAME,
PHONE, EMAIL, ADDRESS, STATUS FROM ORGANIZING_COMMITTEE ORDER BY
COMMITTEE_ID DESC;
  RETURN v_cursor;
END;
```

5. Audit Logs



PLSQL

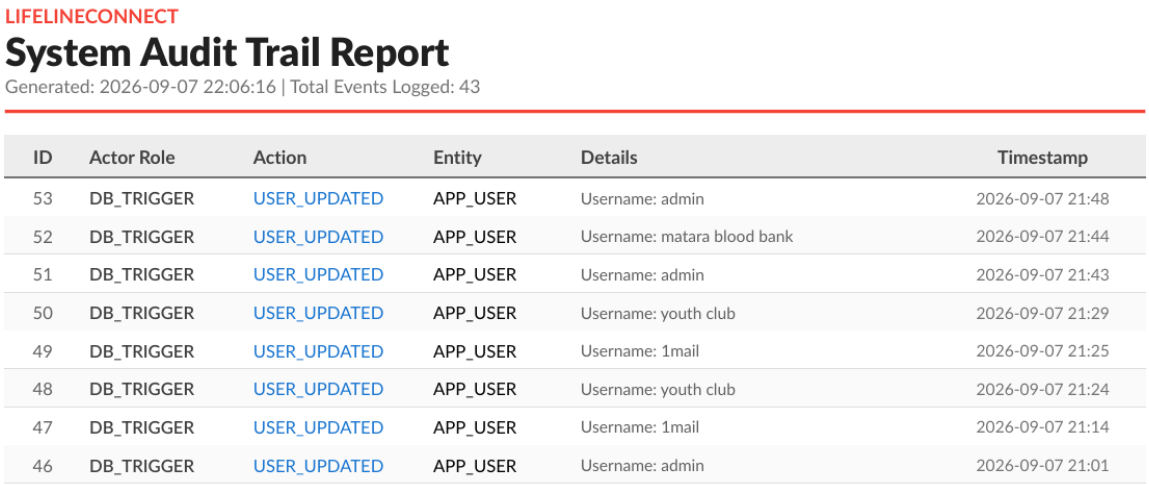
* Uses PL/SQL Procedure: **FN_GET_SYSTEM_AUDIT_LOGS**
* This procedure accepts one input parameter, and it returns related webmaster audit data.

```
CREATE OR REPLACE FUNCTION FN_GET_SYSTEM_AUDIT_LOGS
RETURN SYS_REFCURSOR
IS
    v_cursor SYS_REFCURSOR;
BEGIN
    OPEN v_cursor FOR
    SELECT * FROM (
        SELECT AUDIT_ID, ACTOR_ROLE_CODE, ACTOR_USER_ID, ACTION_CODE,
ENTITY_TYPE, ENTITY_ID, DETAILS, CREATED_AT
        FROM AUDIT_LOG
        ORDER BY CREATED_AT DESC
    ) WHERE ROWNUM <= 200;
    RETURN v_cursor;
END;
```

5. Export Reports

The system provides rich report exporting features for **Audit** Reports, **Bank** Summary, **Committee Summary** and **Users** Summary and can be exported as PDFs.

Example:



LIFELINECONNECT

System Audit Trail Report

Generated: 2026-09-07 22:06:16 | Total Events Logged: 43

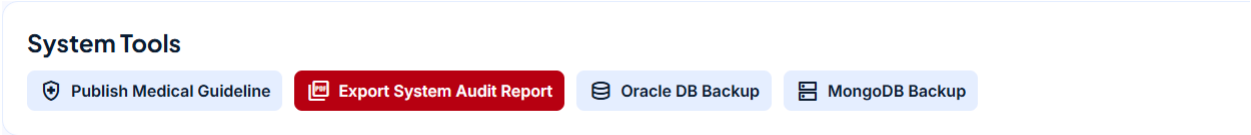
ID	Actor Role	Action	Entity	Details	Timestamp
53	DB_TRIGGER	USER_UPDATED	APP_USER	Username: admin	2026-09-07 21:48
52	DB_TRIGGER	USER_UPDATED	APP_USER	Username: matara blood bank	2026-09-07 21:44
51	DB_TRIGGER	USER_UPDATED	APP_USER	Username: admin	2026-09-07 21:43
50	DB_TRIGGER	USER_UPDATED	APP_USER	Username: youth club	2026-09-07 21:29
49	DB_TRIGGER	USER_UPDATED	APP_USER	Username: 1mail	2026-09-07 21:25
48	DB_TRIGGER	USER_UPDATED	APP_USER	Username: youth club	2026-09-07 21:24
47	DB_TRIGGER	USER_UPDATED	APP_USER	Username: 1mail	2026-09-07 21:14
46	DB_TRIGGER	USER_UPDATED	APP_USER	Username: admin	2026-09-07 21:01

PLSQL

* Uses PL/SQL Functions:
    1. **FN_GET_ALL_USERS**
    2. **FN_GET_ALL_BANKS**
    3. **FN_GET_ALL_COMMITTEES**
    4. **FN_GET_SYSTEM_AUDIT_LOGS**

6. Backup

The system provides access to systemwide database backup features for both MongoDB and Oracle DB.



PLSQL

* Uses PL/SQL Procedures: **GENERATE_SCHEMA_BACKUP**

```
CREATE OR REPLACE PROCEDURE GENERATE_SCHEMA_BACKUP (
    p_dump_file OUT VARCHAR2,
    p_dir_path OUT VARCHAR2
) AUTHID CURRENT_USER
IS
    v_schema VARCHAR2(100);
    v_dp_job NUMBER;
    v_job_state VARCHAR2(30);
BEGIN
    v_schema := SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA');
    p_dump_file := 'LIFELINE_BKP_' || TO_CHAR(SYSDATE, 'YYYYMMDD_HH24MISS')
|| '.dmp';

    SELECT DIRECTORY_PATH INTO p_dir_path
    FROM ALL_DIRECTORIES
    WHERE DIRECTORY_NAME = 'DATA_PUMP_DIR';

    v_dp_job := DBMS_DATAPUMP.OPEN(
        operation => 'EXPORT',
        job_mode => 'SCHEMA',
        remote_link => NULL,
        job_name => NULL,
        version => 'COMPATIBLE'
    );

    DBMS_DATAPUMP.ADD_FILE(
        handle => v_dp_job,
        filename => p_dump_file,
        directory => 'DATA_PUMP_DIR'
    );

    DBMS_DATAPUMP.METADATA_FILTER(
        handle => v_dp_job,
        name => 'SCHEMA_EXPR',
        value => 'IN (''' || v_schema || ''')'
    );

    DBMS_DATAPUMP.START_JOB(v_dp_job);
    DBMS_DATAPUMP.WAIT_FOR_JOB(v_dp_job, v_job_state);
END;
```

7. Pluggable Database

As for a security feature I have also designed the system as a PDB. I have used the Oracle Database Creation Assistant to create the database. This Isolates the whole database, so the main ROOT DB stays safe even after a data breach.

```
ORACLE_DB_CONN="User Id=LLC_ADMIN;Password=password;Data
Source=localhost/LifeLineConnect_PDB;"
```